

ITI113 Progress Check Report

Project Name:

CryptoScam AI — An End-to-End MLOps Pipeline for Cryptocurrency Scam Message Detection

Student name:

Royston Quek Jia Liang (5391216H); Ong Hui Lin (6508864V)

Team:

team03

Submission date:

16 Aug 2026

Table of Content

1. Project Progress Overview.....	3
1.1 Team & Role Allocation.....	3
1.2 Overall Work, Status & Progress.....	3
1.3 Project Notebook Structure and Workflow.....	5
2. Dataset, EDA and Data Quality.....	5
3. Data Preparation and Feature Engineering.....	6
4. Experimental Design and Evaluation Plan.....	6
5. Baseline Models.....	7
6. Model A (Logistic Regression) Development and Initial Results.....	8
7. Model B (Random Forest) Development and Initial Results.....	9
8. Initial Model Comparison and Analysis.....	10
9. MLOps Data and Training Pipeline.....	11
9.1 CryptoScam AI System Architecture Diagram.....	11
9.2 Data Flow & Versioning Mechanism.....	12
9.3 Reproducibility & Audit Trail.....	13
10. Experiment Tracking and Artefact Management.....	13
11. Model Registration / Pipeline Orchestration.....	13
12. Initial Deployment / API test.....	14
13. Final Deployment and Client Plan.....	14
14. AI Risk and Governance Update.....	15
14.1 Bias & Data Issues Identified.....	15
14.2 System-Level Risks (Monitoring, Logging, Control).....	15
15. Issues, Lessons and Next Steps.....	16
16. Individual Contributions / Evidence.....	17
Annex: IMDA ISAGO Simplified Checklist.....	28

1. Project Progress Overview

This project aims to address the growing issue of cryptocurrency scams, which accounted for approximately 20% of Singapore's total scam losses in 2025. The problem is formulated as a supervised binary text-classification task. Machine learning is adopted over keyword-based heuristics as it is more robust to evolving scam language, with the project focusing on interpretable classical models (TF-IDF with logistic regression and random forest). Beyond protecting individual users, automating this first-pass screening also offers an operational efficiency gain over manual review or static keyword blacklists, which don't scale well and require constant updating as scam wording evolves. The target performance metrics are Recall $\geq 85\%$, Precision $\geq 75\%$, F1-score $\geq 80\%$, and ROC-AUC ≥ 0.85 to prioritise detecting scam messages while maintaining acceptable false-positive rates. The final model will be deployed in a Streamlit application to provide scam probability, risk category, and prediction explanations, while directing users to SPF ScamShield and the 1799 anti-scam hotline when approp

1.1 Team & Role Allocation

- Royston Quek — **Modelling**: Model Development (Data & EDA, Modelling)
- Ong Hui Lin — **MLOps Member**: MLOps, Deployment & AI Governance (system-level)

Refer to Section 16 for more details

1.2 Overall Work, Status & Progress

Table 1: Project Implementation Status and Outcomes

Component	Status	Notes
EDA & Data Cleaning	✓ Achieved	The dataset was cleaned with 10,466 rows, 0 missing values, and 0 duplicate messages, and shows a mild class imbalance of 51.5% scam versus 48.5% legitimate that does not require resampling (<i>Notebook 01</i>). Platform-level checks across all 6 source platforms found message counts fairly even, ranging from 1,687 to 1,801 messages each, with scam rates also consistent across platforms at 0.51 to 0.52, so platform does not appear to be a meaningful source of bias in the dataset. Feature engineering produced 13 domain-specific indicator features, and their distributions were reviewed. A multicollinearity check across the engineered features found correlations to be weak to moderate overall, finding directly informed the decision to apply L1/L2 regularisation in the downstream linear model to shrink any remaining redundant coefficients.
Model Development	✓ Achieved	A logistic regression baseline was trained on TF-IDF features combined with the engineered indicators, reaching a test

		AUC-ROC of 0.8898, accuracy of 77.13%, and recall of 76.46%, comfortably clearing the project's 0.85 quality gate (Notebook 02). A 6-candidate hyperparameter sweep over regularisation strength and penalty type found the best configuration at $C = 10.0$ with an L2 penalty, lifting test AUC-ROC only slightly to 0.8908 and recall to 76.74%, while the weakest candidate at $C = 0.01$ collapsed to 0.701 AUC, showing the model needs sufficient regularisation headroom to fit the decision boundary. Random forest was evaluated in parallel as a second model, with its baseline reaching 0.8446 test AUC-ROC and just 56.35% recall, and a 5-candidate tuning sweep over tree count, depth, and leaf size raising AUC-ROC to 0.86 but pushing recall down further to 52.18%, indicating that wider, deeper trees trade recall for precision rather than improving both together. Across all four runs, the tuned logistic regression candidate is the clear leader on every metric.
Data Pipeline	✓ Achieved	Notebook 01 implements SHA256-based versioning with pointer file (_latest.json). TF-IDF vectoriser fitted on training split only (5,000 features, 1 to 2 grams, min_df = 2). Stratified 80/20 train/test split with random_state = 42. All artifacts uploaded to S3.
Experiment Tracking (MLflow)	✓ Achieved	Notebook 01A - Setup Sagemaker MLFlow App complete - SageMaker MLflow App ARN:arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-J5AYUG4AJHVW. Experiment: ITI113/team03/Experiment1. Team tag validation enforced. All baseline and tuning runs logged with metrics and artifacts.
Model Registry	✓ Achieved	Notebook 03 complete. SageMaker Model Registry group: team03-CryptoScamDetector. Registry now at model package v37 across repeated pipeline runs; the currently deployed endpoint uses v37 (Approved). Quality gate enforced: $\text{AUC-ROC} \geq 0.85$. Human-in-the-loop approval required before deployment.
CI/CD (triggered retraining)	✓ Achieved	S3-triggered pipeline (Notebook 05) proven end-to-end: a new data batch in trigger/incoming/ started the pipeline automatically, all four steps succeeded, and model v38 was registered at PendingManualApproval. The polling mechanism watches the trigger folder.

AI Governance (ISAGO checklist)	✓ Achieved	Data versioning via SHA256, model lineage tracked in Registry, audit trail in MLflow, human approval gate before deployment. IMDA ISAGO checklist completed (Refer to <i>Annex: ISAGO Simplified Checklist</i>).
Deployment (Streamlit app)	⚠ Partial	Serverless Endpoint iti113-team03-crypto-scam-detector deployed and InService. Test invocations working (<i>Notebook 04</i>). Streamlit app at developing stage and endpoint integration pending.

1.3 Project Notebook Structure and Workflow

Project Notebooks	Workflow / Notebooks
Notebook 01	01_eda_and_data_preparation_team03.ipynb
Notebook 01A	01A_setup_sagemaker_mlflow_app_team03.ipynb
Notebook 02	02_baseline_experiments_sagemaker_mlflow_app_with_team03.ipynb
Notebook 03	03_sagemaker_pipeline_mlflow_app_with_preprocessing_bundle_team03.ipynb
Notebook 04	04_test_serverless_endpoint_team03.ipynb
Notebook 05	05_s3_trigger_cicd_team03.ipynb

2. Dataset, EDA and Data Quality

Dataset Overview & Data Quality (see Figure 1)

- Dataset name: Crypto Scam Dataset
- Source/Link: <https://www.kaggle.com/datasets/theeyanyan/crypto-scam-dataset/data>
- Target dataset size: 10,466 records
- Data format: Structured tabular dataset (CSV)
- Features: 13 engineered features (see Section 3)
- Target variable: label - scam (51.5%) or legitimate/legit (48.5%)
- No missing values, no duplicate messages

Dataset Schemas

- id: Unique message identifier
- platform: 6 source platforms (e.g., Telegram, X/Twitter, SMS, Email, Discord, Reddit), ranging from 1,687 to 1,801 messages each, with scam rate consistent across platforms at 0.51 to 0.52.
- text: Message content
- label: Binary label (scam or legitimate)

The dataset was synthetically generated. This is expected, as genuine scam data is inherently sensitive and rarely accessible due to privacy constraints, so synthetic data is a practical substitute. It seems like the data was designed to give balanced, linguistically diverse coverage of common scam typologies and everyday legitimate crypto communication, with no real individuals, wallets, or platforms involved.

It's also worth noting that because this is a synthetic dataset rather than real-world scam messages, word patterns and class boundaries may be more linearly separable and more consistent than genuine data would be. The dataset is suitable for learning and demonstration purposes.

3. Data Preparation and Feature Engineering

Raw message text is first normalised, text is lowercased, whitespace is collapsed, and URLs are replaced with a placeholder token rather than removed outright, so their presence still shows up as a normal token in the text without leaking a raw link into the vocabulary. Feature engineering then produces 13 domain-specific engineered features across three categories (see Figure 2-3):

- **Urgency markers** (e.g., exclamation counts, all-caps ratio)
- **Contact information patterns** (e.g., email/phone presence, URL counts)
- **Structural features** (e.g., capital letter ratio)

For the TF-IDF step, we also compare vectorising with and without English stopword removal before deciding on the final configuration. Both approaches produce the full 5,000-term vocabulary, but removing stopwords drops 2,567 terms, 97 of which overlap the scam-related keyword lists used for the engineered features. Since the difference is small but non-zero, the default vectoriser with no stopword removal is used going forward, to avoid discarding any scam-relevant vocabulary. The final TF-IDF vectoriser uses 5,000 features, 1 to 2 gram tokens, and a minimum document frequency of 2, fitted on the training split only to prevent information leakage.

The train/test split is stratified, 80/20, `random_state = 42`, preserving the 51.5% scam, 48.5% legitimate class distribution across both sets.

A multicollinearity check across the 13 engineered features found correlations weak to moderate overall (Figure 4). Findings directly informed the decision to apply L1/L2 regularisation in the downstream linear model to shrink any remaining redundant coefficients.

4. Experimental Design and Evaluation Plan

The evaluation strategy uses a stratified 80/20 train/test split (8,372 training, 2,094 test) with `random_state = 42`, preserving the 51.5% scam / 48.5% legitimate distribution in both splits.

Given the near-balanced classes and the asymmetric cost of scam detection, where a missed scam is costlier than a false alarm, the evaluation suite tracks these complementary metrics:

- **AUC-ROC**, which ranks scams above legitimate messages across all thresholds
- **Accuracy**, a macro-level performance snapshot
- **Recall**, the proportion of actual scams correctly caught, treated as the priority metric given the cost asymmetry

- **Precision**, the proportion of flagged messages that are actually scams
- **F1-score**, the harmonic mean of precision and recall

A quality gate of $AUC \geq 0.85$ is enforced in the Model Registry approval workflow (Notebook 03), so only models meeting minimum discriminative performance are promoted toward production. recall, precision and F1 are reviewed by the team member approving the model package (Section 11) before deployment, so a model that clears the automated gate but falls short on recall as the current best model does is not promoted to the endpoint without a human reviewing the full metric picture.

5. Baseline Models

Logistic regression is used as the baseline model, since it is the simplest model that makes sense for binary text classification. With default parameters, C equal to 1.0, and class_weight set to balanced, it reaches a test AUC-ROC of 0.8898, accuracy of 77.13%, precision of 78.57%, recall of 76.46%, and F1 of 0.7750, comfortably clearing the project's 0.85 quality gate (Figure 5).

Random forest was evaluated in parallel as a second baseline. With 100 estimators, a max depth of 6, a minimum samples leaf of 4, and class_weight set to balanced, it reaches a test AUC-ROC of 0.8446, accuracy of 72.78%, precision of 86.0%, recall of 56.35%, and F1 of 0.6809 (Figure 6). This baseline misses the project's 0.85 quality gate on its own, and its recall is far below logistic regression's.

Table 2: Baseline Model Performance Comparison

Model	Test AUC-ROC	Accuracy	Precision	Recall	F1
Logistic Regression Baseline	0.8898	77.13%	78.57%	76.46%	0.7750
Random Forest Baseline	0.8446	72.78%	86%	56.35%	0.6809

Against the proposal's targets, the logistic regression baseline clears AUC-ROC and precision, but falls short of recall and F1. Random forest clears precision comfortably but falls further short on recall, and does not clear the AUC-ROC gate at all (Figure 7). Both baselines are logged to the SageMaker MLflow App under experiment ITI113/team03/Experiment1, each with a confusion matrix and ROC curve artifact. The random forest run additionally logs a feature importance plot ranked by Gini importance.

6. Model A (Logistic Regression) Development and Initial Results

Model A is built on the shared feature set from Notebook 01, TF-IDF text features with 5,000 features, 1 to 2 gram tokens, and a minimum document frequency of 2, fit on the training split only, combined with the 15 engineered indicator features.

Hyperparameter tuning swept six candidates across regularisation strength C, values of 0.01, 0.1, 1.0, and 10.0, and penalty type, L2 with lbfgs or L1 with liblinear, each evaluated with 5-fold stratified cross-validation.

Table 3: Logistic Regression Candidate Model Comparison

Candidate	C	Penalty	Solver	CV AUC	Test AUC
lr_candidate_01	0.01	L2	lbfgs	0.6896	0.7010
lr_candidate_02	0.1	L2	lbfgs	0.8318	0.8600
lr_candidate_03	1.0	L2	lbfgs	0.8830	0.8899
lr_candidate_04	10.0	L2	lbfgs	0.8857	0.8908
lr_candidate_05	1.0	L1	liblinear	0.8755	0.8867
lr_candidate_06	10.0	L1	liblinear	0.8834	0.8889

The weakest candidate, lr_candidate_01 with C equal to 0.01, collapses to a test AUC of 0.701, showing the model needs enough regularisation headroom to fit the decision boundary. The strongest candidate, lr_candidate_04 with C equal to 10.0 and an L2 penalty, reaches a CV AUC of 0.8857 and a test AUC of 0.8908, only a small step above the untuned baseline.

The selected model, lr_candidate_04, reaches a test AUC-ROC of 0.8908, accuracy of 77.51%, F1 of 0.7786, and recall of 76.74%. This is the best-performing run across all four model and tuning combinations evaluated, and it clears the project's AUC-ROC quality gate of 0.85.

All seven logistic regression runs, the baseline plus six tuning candidates, are logged to the SageMaker MLflow App under experiment ITI113/team03/Experiment1, each with its parameters, metrics, lineage tags and fitted model artifact. Evaluation plots are logged at baseline level: the logistic regression baseline run carries a confusion matrix and ROC curve artifact, while the tuning candidates are compared on logged metrics rather than per-candidate plots.

A recall of 76.7% means roughly one in four scam messages in the test set is missed and classified as legitimate, which is the main quality concern given the project's own framing that missed scams are costlier than false alarms. Precision of about 79% means a modest share of legitimate messages are still flagged as scam.

7. Model B (Random Forest) Development and Initial Results

Model B uses the same feature set as Model A, TF-IDF plus the 15 engineered indicator features.

Hyperparameter tuning swept five candidates over number of estimators, max depth, and minimum samples leaf, with 5-fold stratified cross-validation.

Table 4: Random Forest Candidate Model Comparison

Candidate	Estimators	Max depth	Min sample leaf	CV AUC	Test AUC
rf_candidate_01	50	4	2	0.7800	0.7896
rf_candidate_02	100	4	4	0.7986	0.8219
rf_candidate_03	100	6	2	0.8249	0.8434
rf_candidate_04	200	6	4	0.8213	0.8385
rf_candidate_05	200	8	2	0.8451	0.8600

Performance rose steadily with more trees and greater depth, from a CV AUC of 0.78 for the smallest candidate up to a CV AUC of 0.8451 for the largest, rf_candidate_05, which also reached the best test AUC-ROC of 0.86.

The selected model, rf_candidate_05, reaches a test AUC-ROC of 0.86, accuracy of 73.73%, F1 of 0.6718, and recall of 52.18%. Tuning improved AUC-ROC from 0.8446 to 0.86, but recall fell further, from 56.35% to 52.18%, suggesting that wider, deeper trees trade recall for precision rather than improving both together, this is not ideal for our project.

All six random forest runs, the baseline plus five tuning candidates, are logged to the SageMaker MLflow App under experiment ITI113/team03/Experiment1. As with Model A, evaluation plots are logged at baseline level: the random forest baseline run carries a confusion matrix, ROC curve and a feature importance plot ranked by Gini importance, while the five tuning candidates log parameters, metrics and the fitted model.

For basic error analysis, even the tuned random forest misses close to half of all scam messages in the test set, with recall between 52% and 56% across both runs, well below Model A's roughly 77%. Its precision is higher, between 86% and 94%, so the scams it does flag are rarely wrong. Given the project's cost asymmetry, where missed scams cost more than false alarms, this recall shortfall makes random forest the weaker of the two candidates for our intended goal.

8. Initial Model Comparison and Analysis

Four runs were compared on the current dataset: logistic regression baseline and tuned, random forest baseline and tuned. The table below is ordered by test AUC-ROC.

Table 5: Final Model Performance Comparison

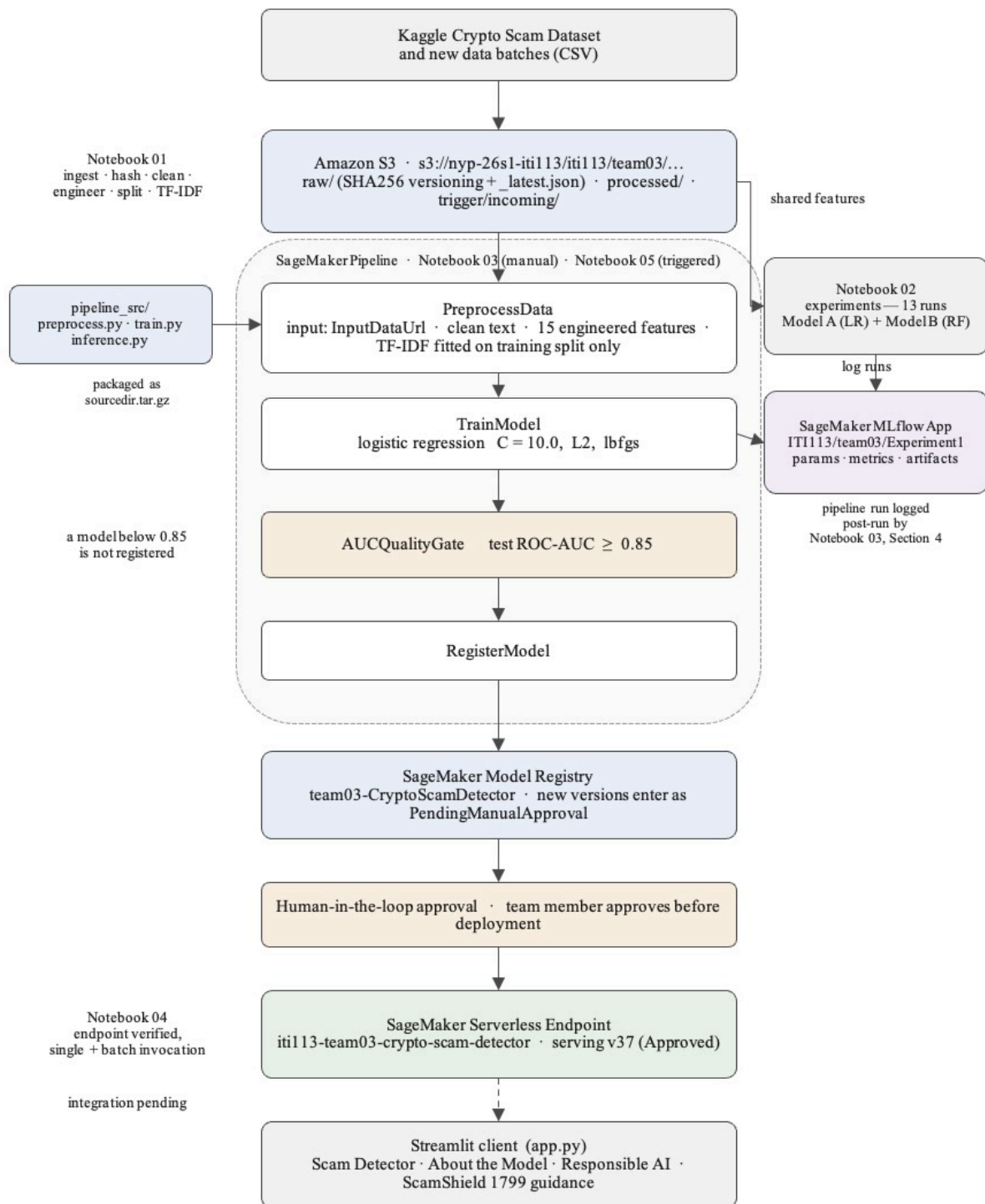
Model	Test AUC-ROC	Accuracy	F1	Recall
Tuned Logistic Regression	0.8908	0.7751	0.7786	0.7674
Baseline Logistic Regression	0.8898	0.7713	0.7750	0.7646
Tuned Random Forest	0.8600	0.7373	0.6718	0.5218
Baseline Random Forest	0.8446	0.7278	0.6809	0.5635

The tuned logistic regression candidate is the overall winner at 0.8908 test AUC-ROC, narrowly ahead of the untuned baseline. Both random forest runs trail on every metric, and the untuned random forest baseline misses the quality gate outright at 0.8446. Random forest's recall, between 52% and 56%, is well below logistic regression's roughly 77%, which matters more than the AUC gap given the proposal's framing that missed scams are more costly than false alarms.

One possible explanation for this gap is the nature of the dataset itself. Since it is synthetically generated rather than drawn from real-world scam messages, word patterns and class boundaries may be more linearly separable and more consistent than genuine data would be, which plays to logistic regression's strengths as a linear model operating on a high-dimensional TF-IDF feature space. Random forest, which partitions the feature space through splits rather than a linear decision boundary, may be less suited to this kind of cleanly separable, high-dimensional sparse text representation, and its lower recall could reflect this mismatch rather than a fundamental weakness of tree-based models for scam detection generally. If this is the case, the gap between the two model families could narrow, or even shift, when evaluated on real-world scam messages, so this comparison should be treated as provisional pending validation on non-synthetic data.

9. MLOps Data and Training Pipeline

9.1 CryptoScam AI System Architecture Diagram



Training and retraining run on AWS. The dashed arrow marks the one path not yet fully automated: the Streamlit client still returns placeholder values pending endpoint integration. (The S3 retraining trigger is also a proof-of-concept polling loop from a notebook rather than an event-driven service, but is not shown as a separate arrow above.)

This figure was created with the assistance of Claude and subsequently reviewed and adapted by the project team.

9.2 Data Flow & Versioning Mechanism

- a. **Raw Data Versioning:** The raw dataset is downloaded from the data source (e.g., Kaggle) each time the pipeline is executed and assigned a SHA256 hash. A pointer file, `raw/_latest.json`, records the latest version number, SHA256 hash, and corresponding S3 key. If the newly downloaded dataset has the same hash as the latest recorded version, the existing version is reused. If the hash differs, the dataset is stored as a new version (`raw/crypto_scam_dataset_v{n}.csv`), and the pointer file is updated accordingly. Previous versions are retained in S3 rather than overwritten, ensuring that the dataset history is preserved and allowing earlier versions to be retrieved and referenced when required.
- b. **S3 organisation:** artifacts are separated by lifecycle stage under `s3://nyp-26s1-iti113/iti113/team03/data/crypto-scam-detector/`

Table 6: S3 Bucket Structure, Contents, and Notebook References

Prefix	Contents	Refer to Notebook
raw/	Versioned source CSVs plus <code>_latest.json</code>	Notebook 01, Section 1
processed/	<code>Train.csv</code> , <code>test.csv</code> , <code>tfidf_vectorizer.joblib</code>	Notebook 01, Section 5
pipeline_src/	<code>preprocess.py</code> , <code>train.py</code> , <code>inference.py</code>	Notebook 03, Sections 1–1B & Notebook 05, Sections 1–1A
pipeline/	Manual pipeline outputs	Notebook 03, Sections 2–3
trigger/incoming/	Watch folder for the retraining trigger	Notebook 05, Sections 3a–3b
pipeline-triggered/	Triggered pipeline outputs	Notebook 05, Sections 2 and 3b

MLflow run artifacts are stored separately at `s3://nyp-26s1-iti113/iti113/team03/mlflow-app-artifacts/`, outside the project prefix, as the MLflow App writes to that location directly.

- c. **Processed Data:** Train/test splits and the fitted TF-IDF vectoriser are stored in `processed/`, fitted on the training split only to prevent leakage into test metrics.

Model A (logistic regression) and Model B (random forest) are both trained in Notebook 02 from these `processed/` artifacts, so they share one feature representation and one train/test split. The SageMaker Pipeline reconstructs an equivalent feature set from `raw/` using the same parameters: `preprocess.py` writes `train_tfidf.npz`, `test_tfidf.npz`, the engineered-feature and label CSVs, and `preprocessor.joblib`. Both `train.py` and `inference.py` read the engineered-feature column order from this bundle rather than assuming a fixed order, so the endpoint reproduces the exact feature ordering used in training.

- d. **Pipeline Source Code:** `preprocess.py`, `train.py` and `inference.py` are written by the notebooks and uploaded to the shared prefix `pipeline_src/`. Both pipelines re-download from this prefix before defining their steps, so each run builds from the S3-held copy rather than the notebook's local `src/` folder. `preprocess.py` and `inference.py` are byte-identical across both pipelines, keeping training and serving consistent; `train.py` differs only in that

Notebook 05's version is simplified to the winning logistic regression configuration. On `pipeline.upsert()`, the SDK packages `pipeline_src/` into a versioned `sourcedir.tar.gz`, which each container pulls at execution time — so code changes take effect on the next upsert.

- e. **Trigger Mechanism:** CSV files placed in `'trigger/incoming/'` automatically invoke the SageMaker Pipeline via a polling loop in Notebook 05 (proof-of-concept; production could use Lambda/EventBridge). Processed input files remain in the watch folder and are tracked in a local seen-keys state file so they are not re-triggered. That state is stored locally rather than in S3, so it does not survive a new Studio instance.

9.3 Reproducibility & Audit Trail

- **Data Lineage:** Lineage is maintained through three linked records rather than a single manifest: (i) `raw/_latest.json` fixes the dataset version and hash; (ii) MLflow logs each run's parameters, metrics and artifacts against `ITI113/team03/Experiment1`; and (iii) the SageMaker Model Registry records each approved version. Notebook 03 Section 4 closes the loop by walking the pipeline execution back to its `TrainModel` step and logging that training job's ARN to MLflow, linking registry version to training run. Notebook 04 walks the serving side in reverse — `endpoint` → `endpoint config` → `model` → `model package`.
- **Source Control & Version Isolation:** Pipeline code is version-controlled in the team's GitHub repository (<https://github.com/evaonghuilin-hub/CryptoScam-AI> - *note: as of 16 Aug, this is a private repository*) (see Figure 8-9 for screenshots), alongside SageMaker's own versioning — `pipeline_src/` archives into a new `sourcedir.tar.gz` on each `upsert()`, S3 raw data versions accumulate under `raw/`, and Model Registry versions increment (currently v38) rather than overwrite — so any past result can be traced back to the exact code and data that produced it, with rollback available if needed.
- **Separation of Concerns:** Raw data, processed features, source code, and execution artifacts are cleanly separated, making the pipeline easier to debug and maintain.

10. Experiment Tracking and Artefact Management

Experiment tracking uses the AWS SageMaker MLflow App. The App was provisioned and configured in `notebooks/01A_setup_sagemaker_mlflow_app_team03.ipynb` (See Figure 10):

All baseline and tuning runs (1 LR baseline, 1 RF baseline, and 6 LR + 5 RF candidates, 13 runs total) are logged under experiment `ITI113/team03/Experiment1`, viewable via the MLflow UI (Figure 11-12). The logistic regression baseline run logs a confusion matrix and ROC curve as artifacts under `plots/` (Figure 13), and the random forest baseline additionally logs a feature importance plot. The best run, `lr_candidate_04` (AUC 0.8908), logs 12 metrics, 6 parameters, lineage tags and its fitted model artifact (Figure 14-15).

11. Model Registration / Pipeline Orchestration

Model versioning is handled entirely by the SageMaker Model Registry, with no separate local registry. The SageMaker Pipelines in Notebook 03, Section 2 (manual, Figure 17-20) and Notebook 05, Section 2 (S3-triggered) both register models to the same Model Registry group, `team03-CryptoScamDetector`, subject to an $\text{AUC-ROC} \geq 0.85$ quality gate (`AUCQualityGate`), matching the threshold specified in the project proposal. A model package must then be approved (`PendingManualApproval` → `Approved`) before deployment, providing a deliberate

human-in-the-loop approval step. Approval considers the full metric set for the run—recall, precision, and F1 alongside AUC-ROC—using the MLflow-logged run (Section 10) or results reported in Section 6. This ensures that models which pass the automated AUC-ROC gate but have weaker recall, as observed in the current best model (Figure 16), have the trade-off explicitly considered rather than being approved based on AUC-ROC alone.

The S3-triggered pipeline (Figure 21) (iti113-team03-crypto-scam-detector-triggered, Notebook 05) demonstrates automated retraining. A watched S3 prefix (trigger/incoming/) is polled for new CSV files, with each new file automatically triggering the pipeline and its S3 URI passed as the `InputDataUrl` parameter. The triggered pipeline runs the same four steps as the manual pipeline and registers a retrained model only if it clears the $\text{AUC-ROC} \geq 0.85$ gate. This was proven end-to-end when a new data batch triggered a successful four-step pipeline execution and registered model v38. The manual pipeline registered v37, which was approved and deployed to the live endpoint.

12. Initial Deployment / API test

The deployed SageMaker Serverless Endpoint (iti113-team03-crypto-scam-detector) was tested directly via boto3 (Notebook 04, Section 3), confirming the endpoint accepts raw message text and returns a prediction, label, and probability (Figure 22-23):

- Request: `{"text": "URGENT: Your wallet has been selected for a guaranteed 100% profit airdrop! Deposit 500 USDT within 1 hour to claim..."}`
 - → Response: Scam, probability 99.9%
- Request: `{"text": "Been dollar-cost averaging into ETH for about a year now, curious what everyone's thoughts are on the current market conditions."}`
 - → Response: Legitimate, probability 10.5%
- Request: `{"text": "Hey, our community wallet is doing a small giveaway this week, check the pinned post in the group for details"}`
 - → Response: Legitimate, probability 20.9%

A batch-invocation test (Notebook 04, Section 4) re-sent these three messages alongside two new ones — an urgent NFT-giveaway message and an unrelated hardware-wallet troubleshooting question — confirming the endpoint handles multiple simultaneous requests via the same request shape (`{"instances": [...]}`). Results were consistent with the single-invocation tests, and correctly separated the urgent NFT message (Scam, 85.8%) from the wallet-related but non-scam technical question (Legitimate, 5.0%).

13. Final Deployment and Client Plan

(Note: Partial/In progress)

The team is developing a serverless, three-tier deployment with a Streamlit web application (app.py) as the client-facing layer. The planned app includes three pages: Scam Detector, About the Model, and Responsible AI.

The Streamlit app is currently under development and returns placeholder results based on keyword counts rather than the deployed model. The SageMaker endpoint integration has been independently implemented and verified in Notebook 04 (`invoke_scam_detector()`), but has not yet been connected to the Streamlit interface. If integration proceeds, the intended flow is: Streamlit UI → SageMaker Serverless Endpoint → `inference.py` → prediction and probability → risk category and guidance → Streamlit display.

14. AI Risk and Governance Update

14.1 Bias & Data Issues Identified

- The dataset is close to balanced, at 51.5% scam and 48.5% legitimate, so no resampling or heavy reweighting was needed. Platform-level scam rates are consistent across the six platforms, at 51 to 52%, so no single platform disproportionately drives scam predictions.
- It is possible that the dataset's synthetic origin introduces its own generalisation and bias concern. If scam and legitimate messages were generated from a smaller set of underlying templates or patterns, a model that performs this well on the dataset may not transfer as effectively to real-world scam messages, which are likely to show greater linguistic variety and creativity. This possibility is factored into the decision to flag deployment as proof-of-concept pending validation on real-world data.
- The engineered indicator features also rely on English-language keyword lists, so a portion of the model's signal is tuned to English scam phrasing conventions. Messages in other languages, or scams that avoid these specific stock phrases, may not trigger these features at all, which is a bias worth flagging for future iterations.

14.2 System-Level Risks (Monitoring, Logging, Control)

- **Access control:** Access to the team's MLflow App is enforced through AWS tag-based IAM rather than shared credentials. The App carries mandatory `course`, `semester`, `teamId`, `studentId` and `projectname` tags (Notebook 01A), and the course IAM policy restricts access by `TeamId`, so `team03`'s experiments are not reachable by other teams. Notebook 01A verifies these tags on every run. UI access is granted through short-lived presigned URLs (300-second expiry) rather than standing credentials, and every run is tagged with the `student_id` that produced it, allowing individual contributions to be audited.
- **Human oversight before deployment:** every model registered by either the manual or triggered pipeline is held at `PendingManualApproval` in the SageMaker Model Registry until a team member explicitly approves it. This is a deliberate control point preventing an automatically retrained model from being silently promoted to the live endpoint. The triggered pipeline's most recent run registered model `v38` (Notebook 05, Section 4) at `PendingManualApproval`, demonstrating that an automatically retrained model is held for human approval rather than promoted to the endpoint.
- **Logging & audit trail:** MLflow captures every training run's parameters and metrics; the SageMaker Model Registry tracks each registered version's approval status. Notebook 03 Section 4 closes the training-side link by logging the pipeline execution's `TrainModel` step, and its originating training job, to MLflow after a successful run; Notebook 04 Section 2

traces the serving side in reverse — endpoint → model package → approval status. Together these give an audit trail from raw data to deployed endpoint.

- **Decision-support framing:** the application is explicitly framed as a decision-support tool rather than a verdict, both in-app (pages/3_Responsible_AI.py: "CryptoShield AI provides a risk assessment rather than a definitive verdict") and in its recommended actions, which direct users to verify through official channels and the ScamShield helpline (1799) rather than act on the tool's output alone.

15. Issues, Lessons and Next Steps

- **Recall Below Target Despite Strong AUC:** Model performance was checked for data leakage given the dataset's strong text-based separability: the TF-IDF vectoriser is fitted on the training split only (Notebook 01, Section 3) and all 15 engineered features are derived purely from message text with no access to the label, so no leakage path was identified in the codebase. The dataset is synthetic, which likely explains why scam and legitimate messages are as separable as they are. The best model (tuned logistic regression, Model A candidate lr_candidate_04) reaches test AUC-ROC 0.8908, clearing the ≥ 0.85 quality gate, but Recall (76.7%) and F1 (0.7786) fall short of the proposal's $\geq 85\%$ and $\geq 80\%$ targets, and random forest's (Model B) recall is lower still at 52–56%.

However, since genuine scam data is inherently sensitive and rarely accessible due to privacy constraints, a synthetic dataset is the practical option available for this project, and this limitation is one we have to accept for now rather than something we can resolve within the current scope.

- **CI/CD Pipeline Proof-of-Concept (S3-Triggered Retraining):** The S3-triggered CI/CD pipeline (Notebook 05) is a proof-of-concept using a Jupyter polling loop and SageMaker Pipeline rather than a production-grade EventBridge/Lambda trigger. This architectural simplification was necessitated by course lab environment constraints (IAM permissions, event-driven service limitations). In a production environment, this would be replaced with an AWS Lambda function triggered by S3 `ObjectCreated` events, eliminating the polling overhead and enabling real-time retraining on new data arrival. The trigger fired and the pipeline completed successfully end-to-end (PreprocessData → TrainModel → AUCQualityGate → RegisterModel), registering model v38 at PendingManualApproval.
- **Remaining Work Before Final Report:**
 - On the MLOps side, the remaining tasks are to integrate the Streamlit client with the deployed SageMaker endpoint and implement a FailStep so that a model failing the quality gate is explicitly marked as a pipeline failure rather than being allowed to complete silently.

16. Individual Contributions / Evidence

Section	Subsection / Description	Student Responsible
1	Project Progress Overview	Group
1.1	Team & Role Allocation	Group
1.2	Overall Work, Status & Progress	Group
1.3	Project Notebook Structure and Workflow	Group
2	Dataset, EDA and Data Quality	Royston
3	Data Preparation and Feature Engineering	Royston
4	Experimental Design and Evaluation Plan	Royston
5	Baseline Models	Royston
6	Model A (Logistic Regression) Development and Initial Results	Royston
7	Model B (Random Forest) Development and Initial Results	Royston
8	Initial Model Comparison and Analysis	Royston
9	MLOps Data and Training Pipeline	Hui Lin
9.1	CryptoScam AI System Architecture Diagram	Hui Lin
9.2	Data Flow & Versioning Mechanism	Hui Lin
9.3	Reproducibility & Audit Trail	Hui Lin
10	Experiment Tracking and Artefact Management	Hui Lin
11	Model Registration / Pipeline Orchestration	Hui Lin
12	Initial Deployment / API test	Hui Lin
13	Final Deployment and Client Plan	Hui Lin
14	AI Risk and Governance Update	Group
14.1	Bias & Data Issues Identified	Royston
14.2	System-Level Risks (Monitoring, Logging, Control)	Hui Lin
15	Issues, Lessons and Next Steps	Group
16	Individual Contributions / Evidence	Royston / Hui Lin
Annex	IMDA ISAGO Simplified Checklist	Group

1. Royston Quek Jia Liang — Model Development & EDA

Evidence below covers the EDA done, as well as model development.

A. Exploratory Data Analysis (Notebook 01)

Figure 1 — Class balance, message count by platform, and scam rate by platform

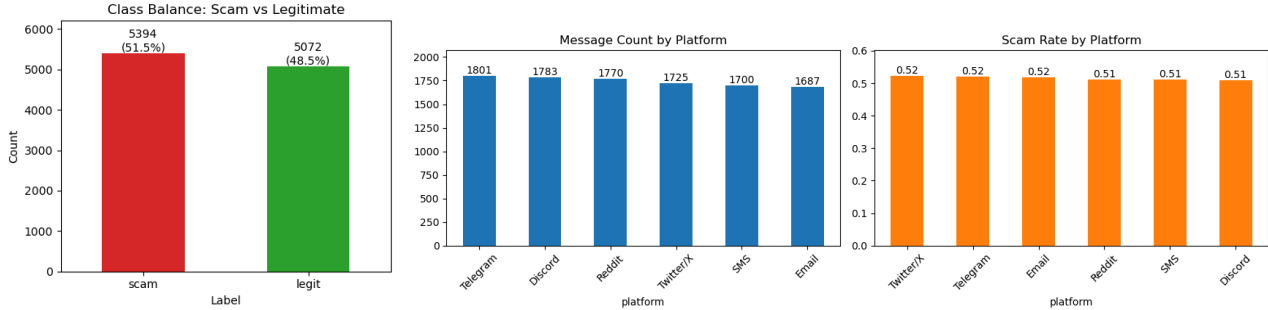


Figure 2 — Text normalisation and feature engineering (sample of engineered features)

urgency_keyword_count	guaranteed_return_keyword_count	countdown_phrase_count	exclamation_count	urgency_score	has_wallet_address	has_url
0	0	0	0	0	0	1
1	0	0	0	0	0	0
2	0	2	1	0	1	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0

Figure 3 — Distribution of engineered features (train set)

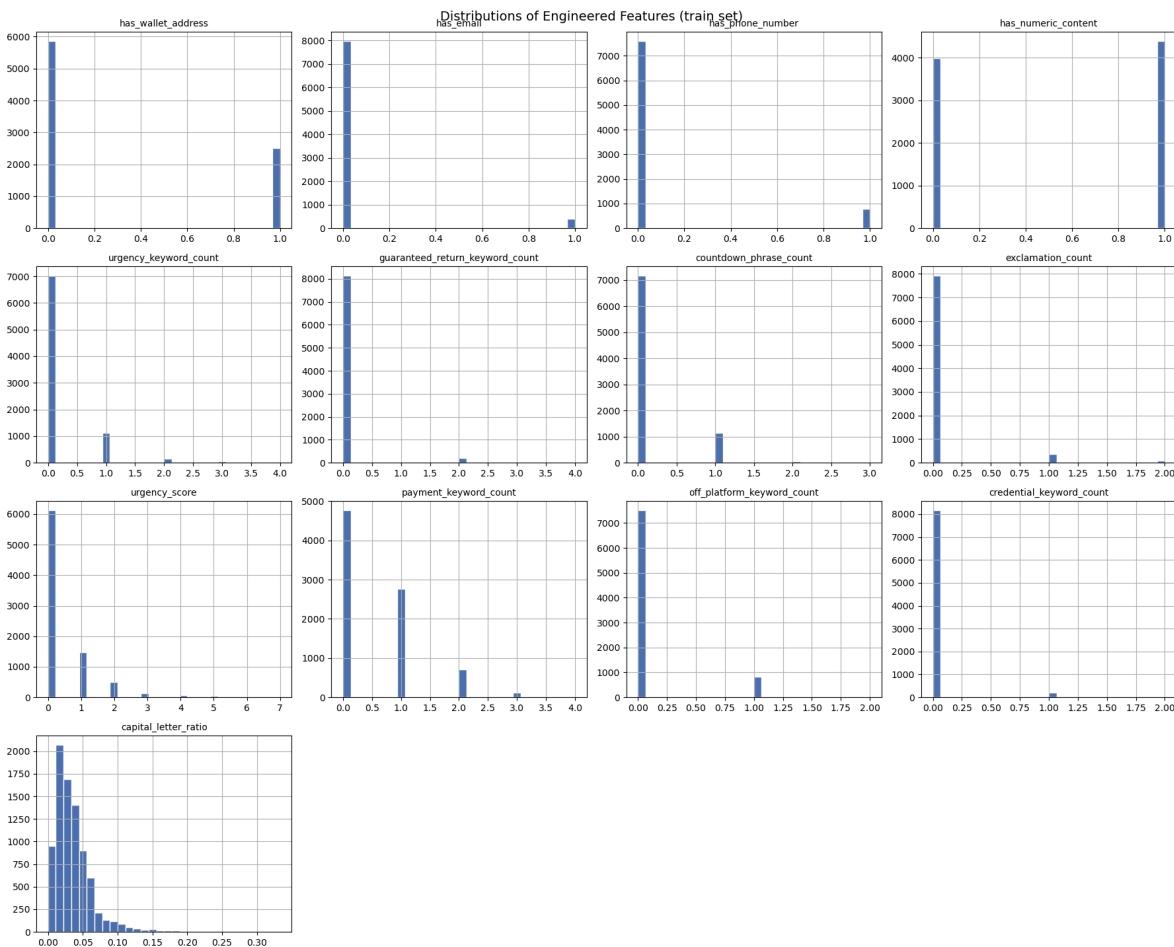
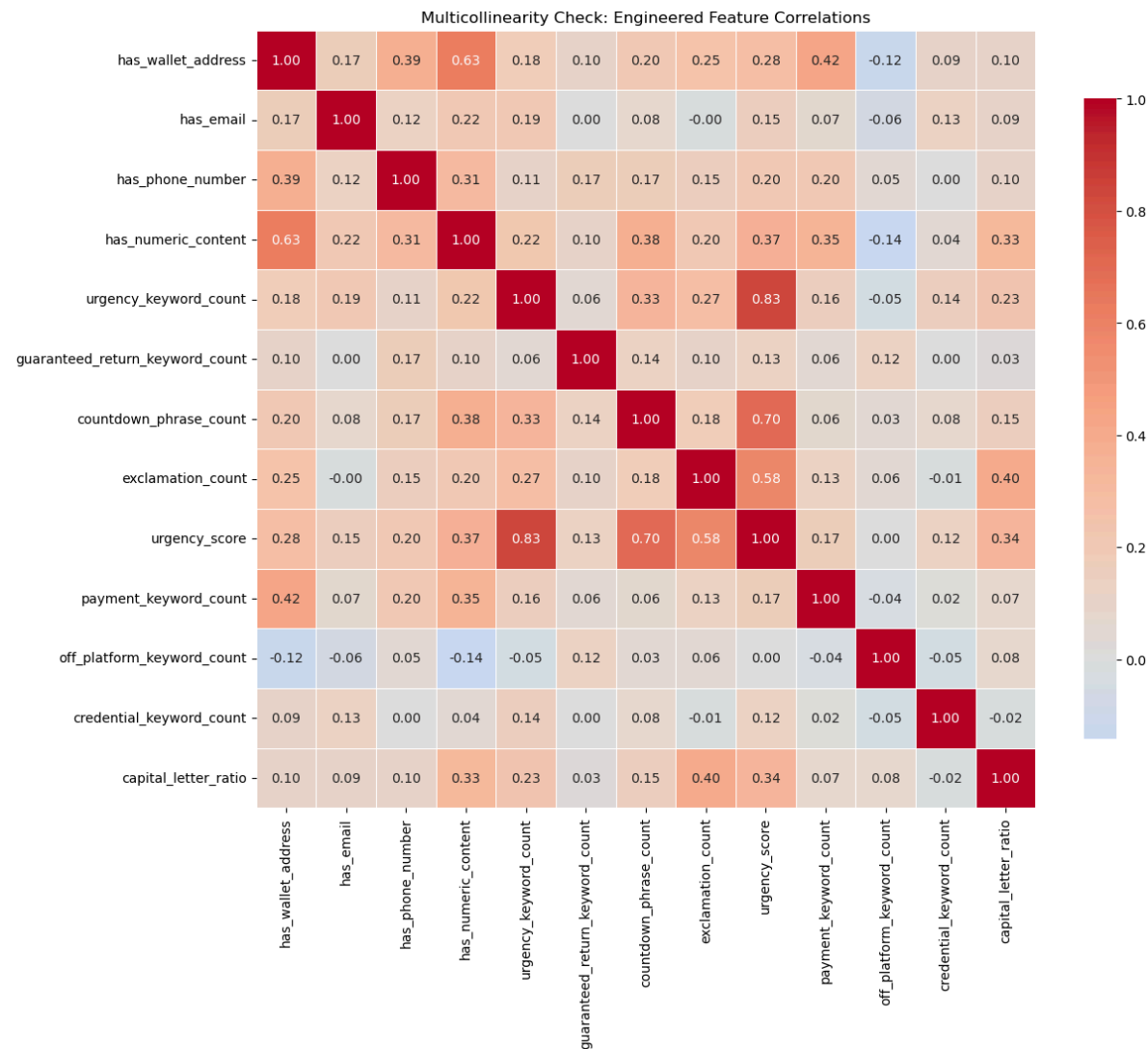


Figure 4 — Multicollinearity check of engineered features (correlation heatmap)



B. Model Development (Notebook 02)

Figure 5 — Confusion matrix and ROC curve for the logistic regression baseline

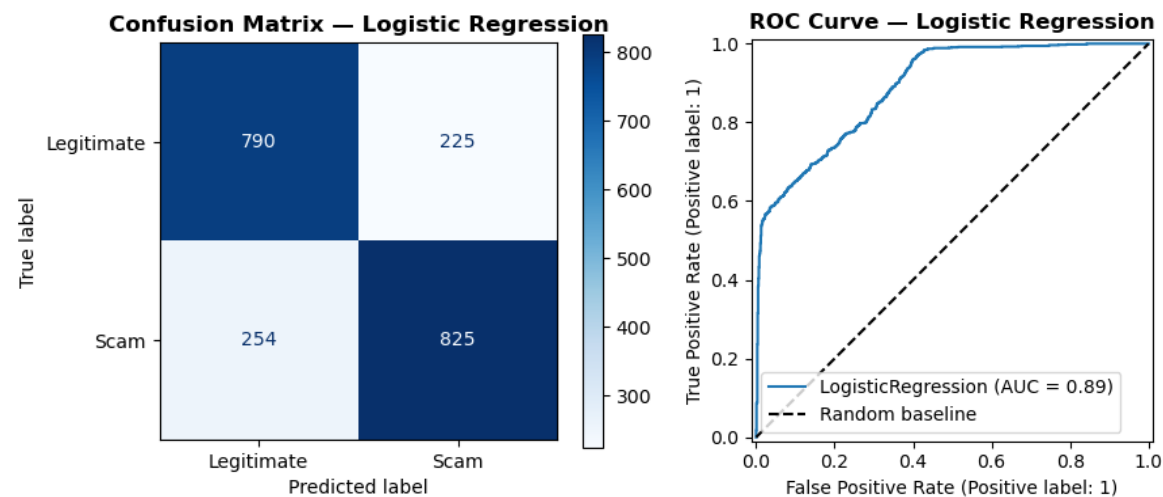


Figure 6 — Confusion matrix and ROC curve for the random forest baseline

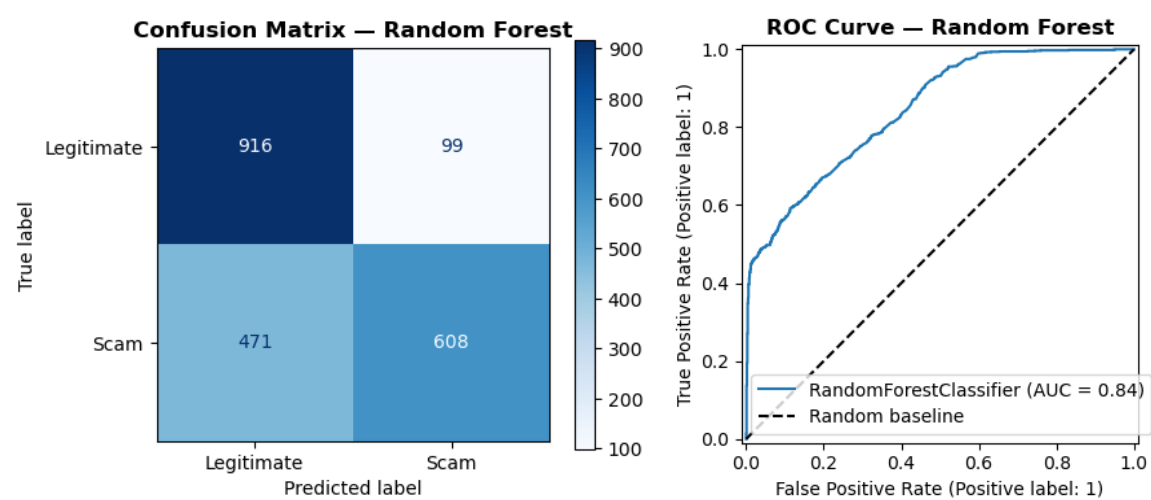
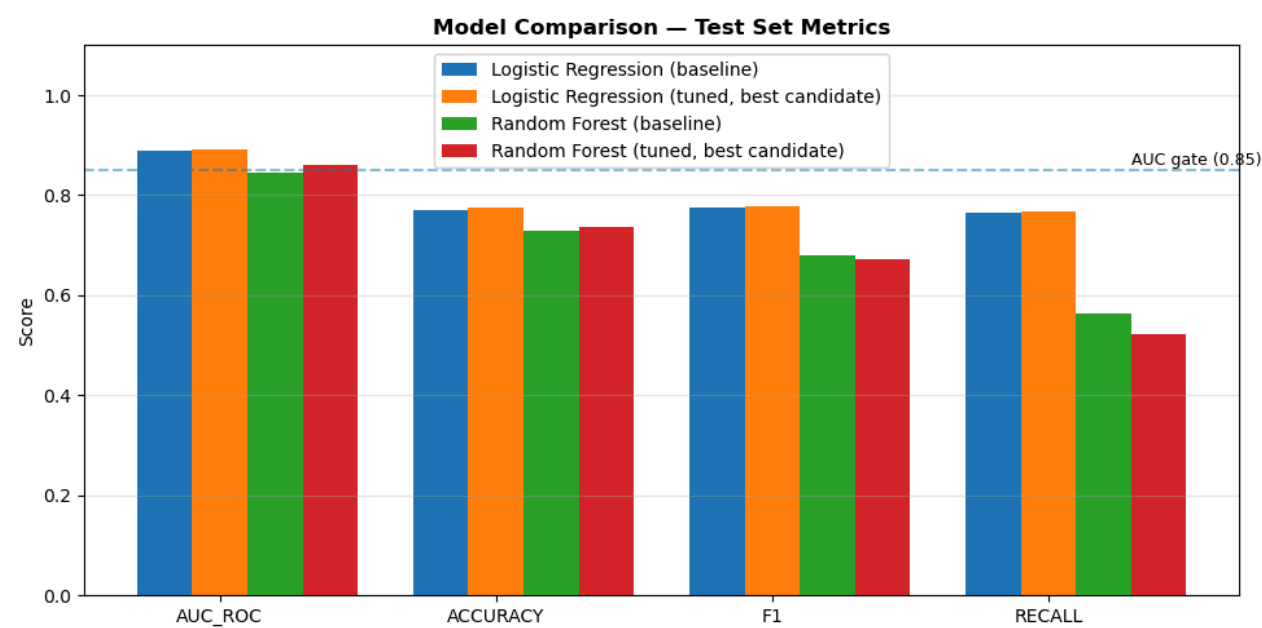


Figure 7 — Test-set metric comparison across baseline and tuned models



2. Ong Hui Lin — MLOps, Deployment & AI Governance

Evidence below covers the experiment-tracking platform, data pipeline, SageMaker Pipeline, Model Registry, serverless endpoint and CI/CD trigger.

A. Version Control & Traceability (GitHub)

Figure 8 — GitHub commit history for notebooks/, showing the traceable sequence of pipeline updates and re-runs contributed by both team members across the project timeline.

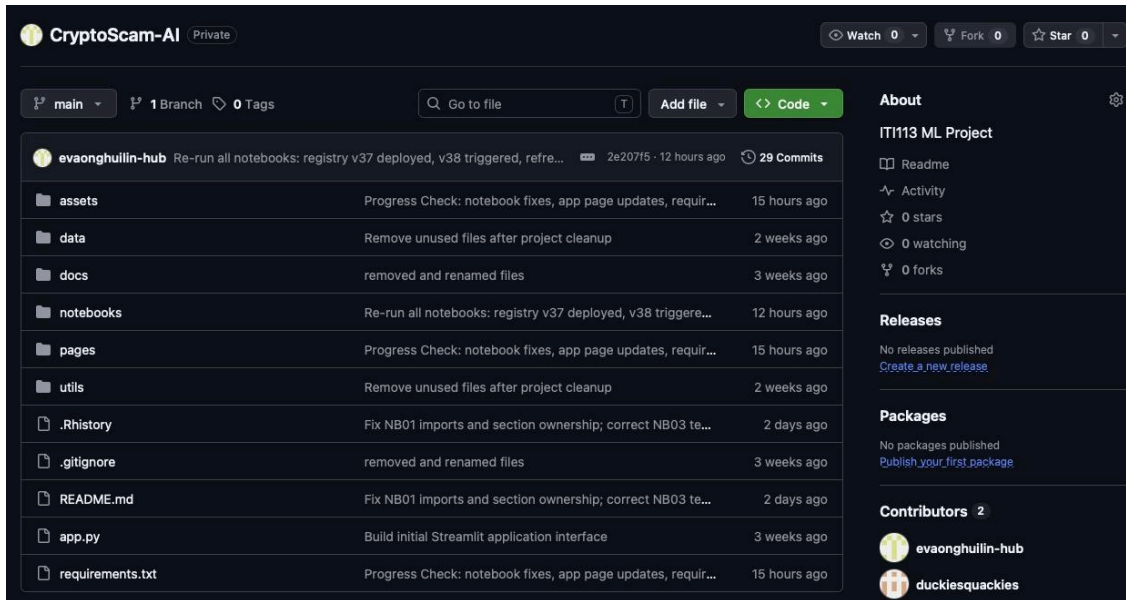
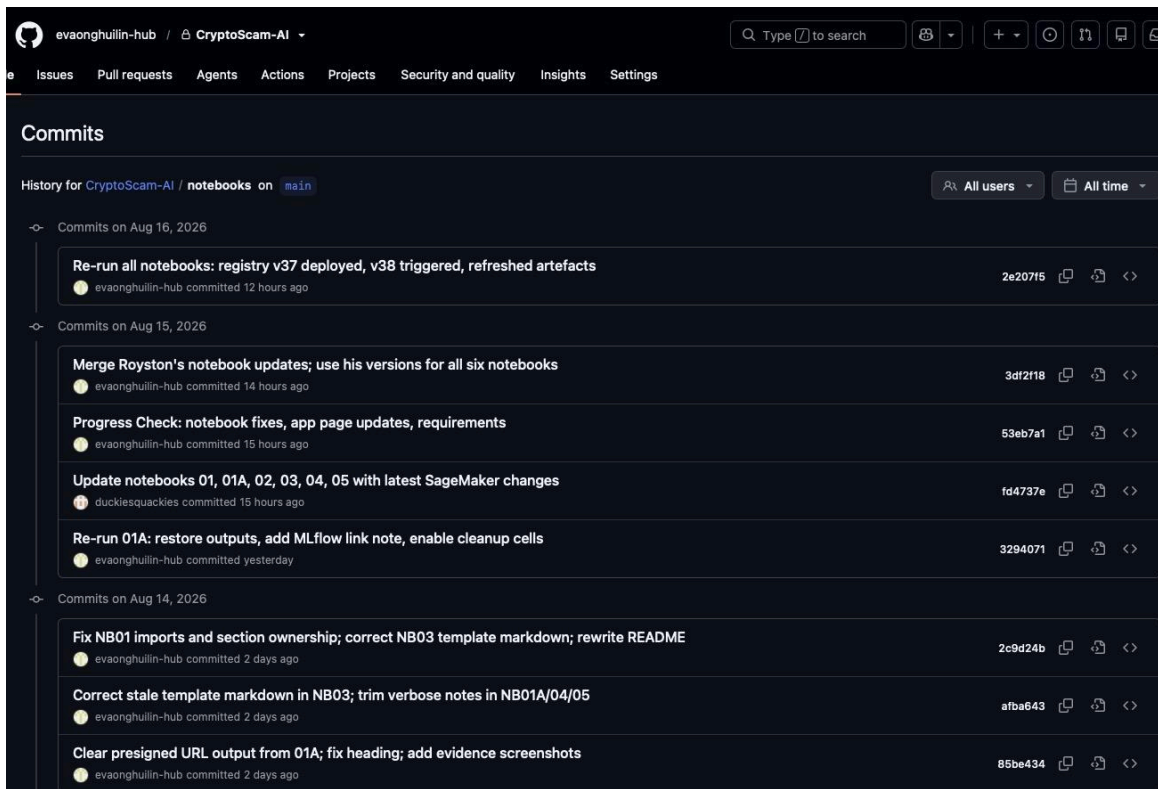


Figure 9 — CryptoScam-AI repository structure on GitHub (notebooks, pages, utils, app.py, requirements.txt), with 29 commits and 2 contributors, evidencing the source control referenced in Section 9.3.



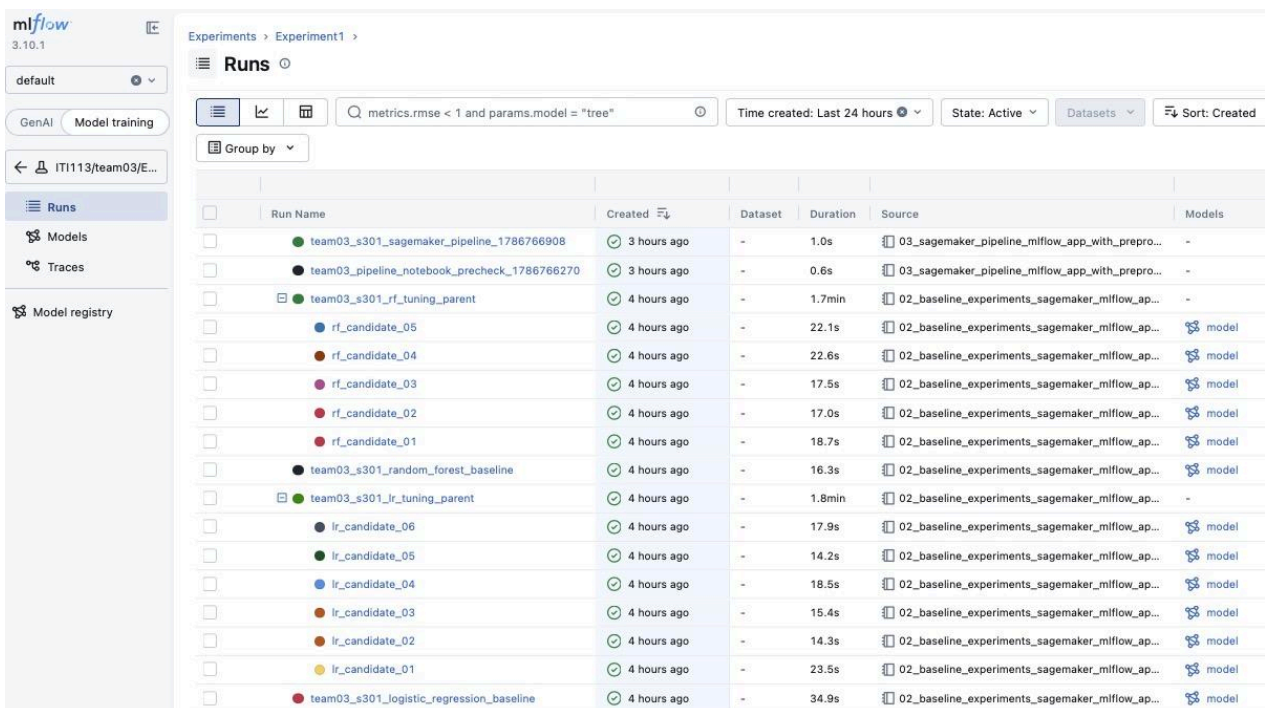
B. Experiment tracking platform (data-pipeline sections of Notebook 01A)

Figure 10 — MLflow App configuration exported by Notebook 01A and reused by Notebooks 02, 03 and 05, giving one tracking target for the whole project.

```
REGION = "ap-southeast-1"
MLFLOW_APP_ARN = "arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-J5AYUG4AJHVW"
EXPERIMENT_NAME = "ITI113/team03/Experiment1"
TEAM_ID = "team03"
STUDENT_ID = "s301"
PROJECT_NAME = "crypto-scam-detector"

Saved local config: mlflow_app_config_team03_s301.json
```

Figure 11 — Experiment ITI113/team03/Experiment1: 13 runs (1 LR baseline, 1 RF baseline, 6 LR + 5 RF candidates) runs across both modelling tracks, with tuning candidates nested under parent runs.



Run Name	Created	Dataset	Duration	Source	Models
team03_s301_sagemaker_pipeline_1786766908	3 hours ago	-	1.0s	03_sagemaker_pipeline_mlflow_app_with_prepro...	-
team03_pipeline_notebook_precheck_1786766270	3 hours ago	-	0.6s	03_sagemaker_pipeline_mlflow_app_with_prepro...	-
team03_s301_rf_tuning_parent	4 hours ago	-	1.7min	02_baseline_experiments_sagemaker_mlflow_ap...	-
rf_candidate_05	4 hours ago	-	22.1s	02_baseline_experiments_sagemaker_mlflow_ap...	model
rf_candidate_04	4 hours ago	-	22.6s	02_baseline_experiments_sagemaker_mlflow_ap...	model
rf_candidate_03	4 hours ago	-	17.5s	02_baseline_experiments_sagemaker_mlflow_ap...	model
rf_candidate_02	4 hours ago	-	17.0s	02_baseline_experiments_sagemaker_mlflow_ap...	model
rf_candidate_01	4 hours ago	-	18.7s	02_baseline_experiments_sagemaker_mlflow_ap...	model
team03_s301_random_forest_baseline	4 hours ago	-	16.3s	02_baseline_experiments_sagemaker_mlflow_ap...	model
team03_s301_lr_tuning_parent	4 hours ago	-	1.8min	02_baseline_experiments_sagemaker_mlflow_ap...	-
lr_candidate_06	4 hours ago	-	17.9s	02_baseline_experiments_sagemaker_mlflow_ap...	model
lr_candidate_05	4 hours ago	-	14.2s	02_baseline_experiments_sagemaker_mlflow_ap...	model
lr_candidate_04	4 hours ago	-	18.5s	02_baseline_experiments_sagemaker_mlflow_ap...	model
lr_candidate_03	4 hours ago	-	15.4s	02_baseline_experiments_sagemaker_mlflow_ap...	model
lr_candidate_02	4 hours ago	-	14.3s	02_baseline_experiments_sagemaker_mlflow_ap...	model
lr_candidate_01	4 hours ago	-	23.5s	02_baseline_experiments_sagemaker_mlflow_ap...	model
team03_s301_logistic_regression_baseline	4 hours ago	-	34.9s	02_baseline_experiments_sagemaker_mlflow_ap...	model

C. Model Selection - MLFlow Focused (Notebook 02)

Figure 12 — Baseline comparison in MLFlow: logistic regression 0.890 test ROC-AUC against random forest 0.845.



Figure 13 — Logistic regression baseline: confusion_matrix.png and roc_curve.png logged as artifacts under plots/ (AUC 0.89)

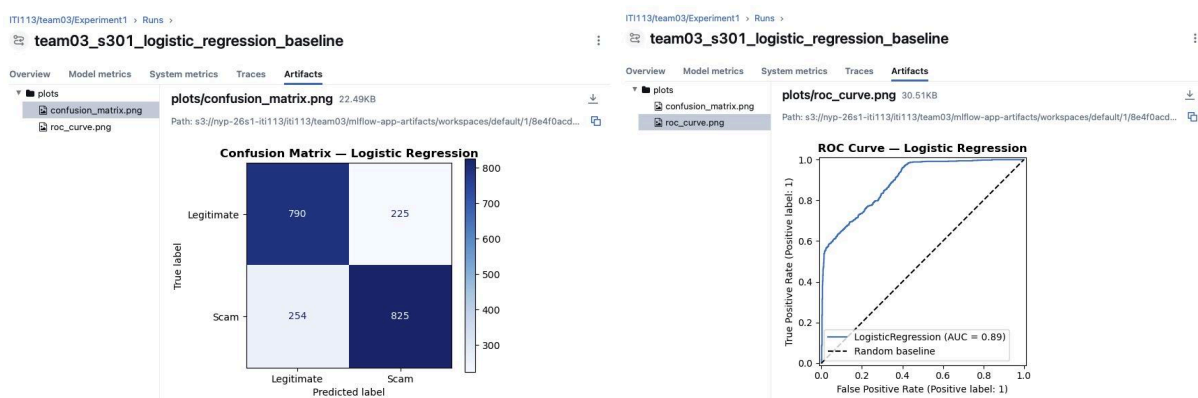


Figure 14 — logistic regression tuning sweep in MLFlow: six candidates over C and penalty type; lr_candidate_04 (C=10.0, L2) is best at 0.891.

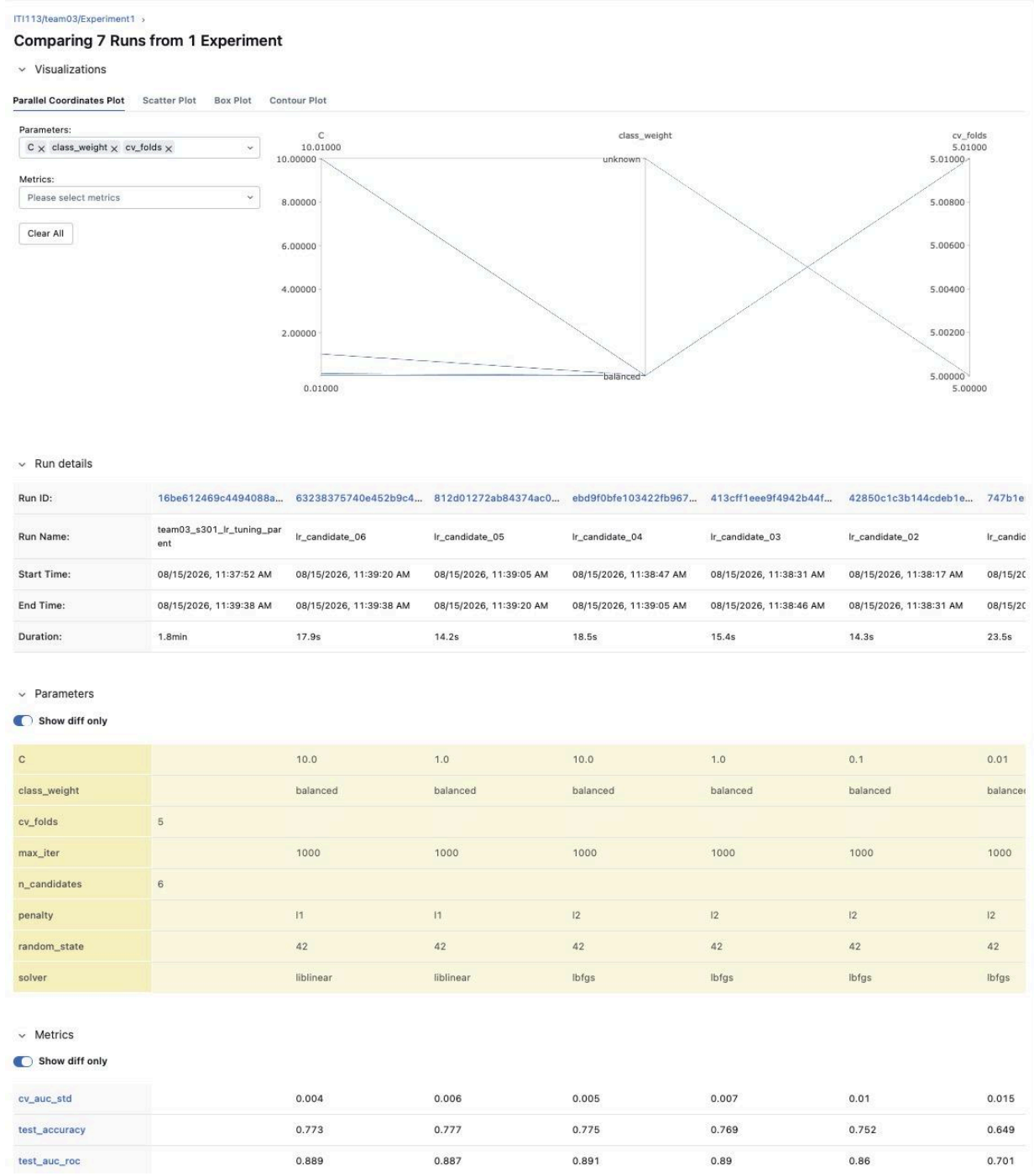


Figure 15 — Selected model lr_candidate_04: 12 logged metrics, 6 parameters, and tags recording team, student and dataset SHA256 (31e13ac65ebc) for lineage.

ITI113/team03/Experiment1 > Runs > **lr_candidate_04**

Overview Model metrics System metrics Traces Artifacts

Description [✎](#)
No description

Metrics (12)

Metric	Value	Models
cv_auc_mean	0.8857	model
cv_auc_std	0.0046	model
train_accuracy	0.8327	model
train_f1	0.8333	model
train_precision	0.8564	model
train_recall	0.8114	model
train_auc_roc	0.9324	model
test_accuracy	0.7751	model
test_f1	0.7786	model
test_precision	0.7901	model
test_recall	0.7674	model
test_auc_roc	0.8908	model

Parameters (6)

Parameter	Value
C	10.0
penalty	l2
solver	lbfgs
max_iter	1000
class_weight	balanced
random_state	42

Logged models (1)

Model attributes				
Type	Step	Model name	Status	Created
Output	0	model	Ready	9 hours ago

About this run

Created at 08/15/2026, 11:38:44 PM
 Created by [sagemaker-user](#)
 Experiment ID [1](#)
 Status [Finished](#)
 Run ID [3fe4e665942a4961a467c0d4f43e00d3](#)
 Duration 14.7s
 Parent run [team03_s301_lr_tuning_parent](#)
 Source [02_baseline_experiments_sagemaker_mlflow_app_with_team03.ipynb](#)
 Registered prompts —

Datasets
None

Tags
 course: ITI113 semester: 26S1 team_id: team03 student_id: s301
 team: team03 student: s301 dataset: crypto-scam-detector
 dataset_sha256: 31e13ac65ebc
 tracking_backend: sagemaker_mlflow_app
 mlflow_app_arn: arn:aws:sagemaker:ap-south-1:044528205969:mlflow-app/app-J5AYUG4AJHWV
 run_session_id: 20260815T153718Z-fd846aae [✎](#)

Registered models
None

Figure 16 — best_model.json written by Notebook 02 and read by Notebook 03 Section 0B, so the pipeline trains whichever model won the comparison.

```
{
  "team_id": "team03",
  "student_id": "s301",
  "project_name": "crypto-scam-detector",
  "best_run_id": "3fe4e665942a4961a467c0d4f43e00d3",
  "best_run_name": "lr_candidate_04",
  "best_auc": 0.8908,
  "best_model_uri": "runs:/3fe4e665942a4961a467c0d4f43e00d3/model",
  "mlflow_tracking_uri": "arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-J5AYUG4AJHWV",
  "mlflow_experiment": "ITI113/team03/Experiment1",
  "tracking_backend": "sagemaker_mlflow_app",
  "mlflow_app_team_tag": "team03"
}
```

D. Pipeline Orchestration (Notebook 03)

Figure 17 — Manual pipeline execution started.

```
Execution ARN: arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/iti113-team03-crypto-scam-detector/execution/spwqpzn7wm97
```

Figure 18 — Manual pipeline: PreprocessData → TrainModel → AUCQualityGate → RegisterModel, all Succeeded.

```
PreprocessData      Executing
TrainModel          Executing
PreprocessData      Succeeded
RegisterModel-RepackModel=0 Executing
AUCQualityGate      Succeeded
TrainModel          Succeeded
RegisterModel-RegisterModel Succeeded
RegisterModel-RepackModel=0 Succeeded
```

Pipeline Status: Succeeded

Figure 19 — Captured metrics (test ROC-AUC 0.890) passed to the quality gate, and the completed run logged to the MLflow App.

```
Pipeline steps:
RegisterModel-RegisterModel: Succeeded
RegisterModel-RepackModel=0: Succeeded
AUCQualityGate: Succeeded
TrainModel: Succeeded
PreprocessData: Succeeded
Training job: pipelines-q6bopx89otod-TrainModel-x2MZjq5byE
Model artefact: s3://sagemaker-ap-southeast-1-044528205969/pipelines-q6bopx89otod-TrainModel-x2MZjq5byE/output/model.tar.gz
Captured metrics: {'test_auc_roc': 0.8899999856948853, 'test_accuracy': 0.7713000178337097, 'test_f1': 0.775600016117096}
🔗 View run team03_s301_sagemaker_pipeline_1786809164 at: https://mlflow.sagemaker.ap-southeast-1.app.aws/#/experiments/1/runs/4527ced4ae4849b99e69d7c753d4ee48
📄 View experiment at: https://mlflow.sagemaker.ap-southeast-1.app.aws/#/experiments/1
SageMaker MLflow App logging completed.
MLflow run ID: 4527ced4ae4849b99e69d7c753d4ee48
MLflow App ARN: arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-J5AYUG4AJHW
Experiment: ITI113/team03/Experiment1
Experiment ID: 1
```

Figure 20 — Post-pipeline MLflow run linking the SageMaker training job, pipeline execution ARN and model artefact back to the experiment.

ITI113/team03/Experiment1 > Runs >

team03_s301_sagemaker_pipeline_1786766908

Overview | Model metrics | System metrics | Traces | Artifacts

Description

No description

Metrics (3)

Metric	Value
test_auc_roc	0.8899999856948853
test_accuracy	0.7713000178337097
test_f1	0.775600016117096

Parameters (17)

Parameter	Value
max-depth	8
max-iter	1000
min-samples-leaf	2
model-type	logistic_regression
n-estimators	200
penalty	l2
random-state	42
reg-c	10.0
run-name	"sagemaker_pipeline_run"
sagemaker_container_log_level	20
sagemaker_program	"train.py"
sagemaker_region	"ap-southeast-1"
sagemaker_submit_directory	"s3://sagemaker-ap-southeast-1-044528205969/iti113-team03-s301-train-2026-08-15-03-57-53-399/source/sourcedir.tar.gz"

About this run

Created at: 08/15/2026, 12:08:28 PM
Created by: sagemaker-user
Experiment ID: 1
Status: 🟢 Finished
Run ID: 243b8c3803d4488e975f0066b093ac61
Duration: 1.0s
Source:
03_sagemaker_pipeline_mlflow_app_with_preprocessing_bundle_team03_v10.ipynb

Logged models
Registered prompts: —

Datasets

None

Tags

course: ITI113 semester: 2651 team_id: team03 student_id: s301
dataset: crypto-scam-detector
execution_environment: aws_sagemaker_pipeline
tracking_backend: sagemaker_mlflow_app
pipeline_arn: arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/iti113-team03-crypto-scam-detector/execution/spwqpzn7wm97
training_job_arn: pipelines-yyz4zg5vh1s-TrainModel-x2MZjq5byE
sagemaker_model_arn: s3://sagemaker-ap-southeast-1-044528205969/pipelines-yyz4zg5vh1s-TrainModel-x2MZjq5byE/output/model.tar.gz
mlflow_app_arn: arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-J5AYUG4AJHW
mlflow_experiment_id: ITI113/team03/Experiment1

Registered models

None

E. Deployment (Notebook 04)

Figure 21 — S3-triggered CI/CD run (Notebook 05): new file detected in trigger/incoming/, pipeline started automatically, all four steps Succeeded, model v38 registered at PendingManualApproval.

```
Latest execution: arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/iti113-team03-crypto-scam-detector-triggered/execution/gd3l0bpinwq
Status: Succeeded

Step-by-step status:
RegisterModel-RegisterModel: Succeeded
RegisterModel-RepackModel-0: Succeeded
AUCQualityGate: Succeeded
TrainModel: Succeeded
PreprocessData: Succeeded
Most recent model in team03-CryptoScamDetector:
Model version: 38
Status: Completed
Approval status: PendingManualApproval
```

Figure 22 — Serverless endpoint iti113-team03-crypto-scam-detector InService, serving model package team03-CryptoScamDetector/37 (Approved).

```
Endpoint name: iti113-team03-crypto-scam-detector
Endpoint status: InService
Endpoint config: iti113-team03-crypto-scam-detector
Variant name: AllTraffic
Model name: team03-CryptoScamDetector-2026-08-15-15-52-46-126
Model package: arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/team03-CryptoScamDetector/37
Model package status: Approved
```

Figure 23 — Endpoint invocation test: scam-style message returns Scam at 99.9%, legitimate-style message returns Legitimate at 10.5%, borderline/ambiguous message returns Legitimate at 20.9%.

3. Test Endpoint Invocation with boto3

The endpoint accepts JSON input with the raw message text, the same shape pages/1_Scam_Detector.py will send: a text field holding the message string. It returns a list with one object containing prediction, label and probability. inference.py from Notebook 03 performs the cleaning, engineered-feature extraction and TF-IDF transform internally, so callers only ever send raw text.

```
def invoke_scam_detector(text, endpoint_name=ENDPOINT_NAME):
    """Invoke the deployed crypto-scam-detector endpoint with a single raw message.

    This is the exact call pages/1_Scam_Detector.py makes once wired up, validated here first and then reused in the Streamlit app.
    """
    response = runtime.invoke_endpoint(
        EndpointName=endpoint_name,
        ContentType="application/json",
        Body=json.dumps({"text": text}),
    )
    return json.loads(response["Body"].read())[0]

scam_message = (
    "URGENT: Your wallet has been selected for a guaranteed 100% profit airdrop! "
    "Deposit 500 USDT to your wallet address within 1 hour to claim now. "
    "Contact us on Telegram immediately, don't miss out!"
)

result = invoke_scam_detector(scam_message)
print(f"SCAM-STYLE MESSAGE\nPrediction: {result['label']}\nProbability: {result['probability']:.1%}")

SCAM-STYLE MESSAGE
Prediction: Scam
Probability: 99.9%

legit_message = (
    "Been dollar-cost averaging into ETH for about a year now, curious what "
    "everyone's thoughts are on the current market conditions."
)

borderline_message = (
    "Hey, our community wallet is doing a small giveaway this week, check the "
    "pinned post in the group for details."
)

for label, msg in [("LEGIT-STYLE MESSAGE", legit_message), ("BORDERLINE / AMBIGUOUS MESSAGE", borderline_message)]:
    result = invoke_scam_detector(msg)
    print(f"{label}\nPrediction: {result['label']}\nProbability: {result['probability']:.1%}\n")

LEGIT-STYLE MESSAGE
Prediction: Legitimate
Probability: 10.5%

BORDERLINE / AMBIGUOUS MESSAGE
Prediction: Legitimate
Probability: 20.9%
```

Annex: IMDA ISAGO Simplified Checklist

ITI113 Machine Learning & Operations Project

Ethical AI System Implementation Guidelines

Project Name: CryptoScam AI — An End-to-End MLOps Pipeline for Cryptocurrency Scam Message Detection

Team Number: 03

Student ID	Student Name
5391216H	Royston Quek Jia Liang
6508864V	Ong Hui Lin

1. Introduction

This assessment is based on (1) the Model AI Governance Framework (2nd Edition) and (2) Implementation and Self-Assessment Guide for Organizations (“ISAGO”) published by Info-Communications Development Authority (IMDA) & Personal Data Protection Commission (PDPC).

- **Model AI Governance Framework**¹ is a general, ready-to-use tool to enable organizations that are deploying AI solutions at scale to do so in a responsible manner.
- **ISAGO Implementation Guide**² helps organizations assess the alignment of their AI governance practices and processes with the Model Framework.

Only 11 of the 64 guiding questions from ISAGO were used in this assessment questionnaire and it is meant for students doing AI-related projects in their studies.

Three additional columns have been added. The Response column allows the project team to specify whether the questions have been responded to or are not applicable to the project. The details of the response are included under the Remarks column. The Stage column suggests when the questions are most applicable throughout the project life cycle.

Definition of Stages:

1. Initiation Stage – Initial Problem Definition
2. Implementation Stage – AI Model Prototyping & Testing and Prototype / Project Development

2. System Profile

Project Name	CryptoScam AI — An End-to-End MLOps Pipeline for Cryptocurrency Scam Message Detection
Project Description	Cryptocurrency-related scams are an increasing global and local concern, as fraudsters exploit digital messaging platforms to reach victims quickly and at scale. Despite growing awareness, users often lack timely and reliable support to assess whether crypto-related messages are fraudulent before taking action. To address this gap, the project frames cryptocurrency scam detection as a supervised binary text-classification problem, where the input is a crypto-related message and the output is a binary label indicating scam or legitimate. By automatically distinguishing scam messages from legitimate crypto-related communications, the project aims to provide early, decision-support assistance to users using a data-driven and interpretable machine learning (ML) approach.

¹ Model AI Governance Framework available at <http://go.gov.sg/AI-gov-MF-2>

² ISAGO available at <http://go.gov.sg/ISAGO>

3. Assessment Questionnaire

	Guiding Question		Response	Stage	Remarks
Section 1: Objectives of deploying AI					
1.	Has your project team defined a clear purpose in using the identified AI solution (e.g. operational efficiency and cost reduction)?		☒ YES ☐ NO ☐ NA	Initiation	Our project team has defined a clear purpose for the AI solution to provide early, decision-support assistance to users who receive suspicious cryptocurrency-related messages. Users often lack timely, reliable ways to assess whether a message is fraudulent before acting. Beyond user protection, the solution also targets operational efficiency. A rule-based or manual review approach (e.g. keyword blacklists, human triage of reported messages) does not scale well and requires constant manual updating as scam wording evolves. By automating the initial risk assessment through a supervised ML model, the system can screen messages instantly and at scale, reducing the manual effort needed to verify each message before a user or a support channel has to get involved. This makes the AI solution both a user-safety measure and an operational efficiency gain, since it cuts down the time and effort spent on first-pass scam triage compared to purely manual or static rule-based approaches. Also, its value is therefore not dependent on meeting only a single performance target; even where a model falls short of a specific proposal target, such as the recall shortfall (discussed in section 15), it can still support faster and more consistent first-pass triage than purely manual or static rule-based approaches, provided that its outputs remain for decision support rather than autonomous action. The four proposal metrics: recall, precision, F1 and ROC-AUC are therefore used as quality thresholds for assessing whether a model version is suitable for deployment, rather than as the primary reason for adopting an AI-based approach in the first place.

Section 3: Determining the level of human involvement in AI-augmented decision-making				
2.	Did your project team implement the appropriate level of human involvement in AI-augmented decision-making?		☒ YES ☐ NO ☐ NA	Implementation We implemented a human-over-the-loop approach at two levels. First, at the model lifecycle level, every retrained model is held at PendingManualApproval in the SageMaker Model Registry until a team member explicitly approves it (Notebook 03), preventing an automatically retrained model from being silently promoted to the live endpoint. Second, at the application level, the Streamlit app is explicitly framed as a decision-support tool rather than a verdict, and directs users to verify suspicious messages through official channels such as the ScamShield hotline (1799) before taking any action.
3.	[Relevant only in limited scenarios]: For safety-critical systems, did your project team ensure that: A. The relevant personnel will be able to assume control where necessary? B. The AI solution provides sufficient information to assist the personnel to make an informed decision and take actions accordingly?		☐ YES ☐ NO ☒ NA	Implementation This project is not a safety-critical system. It is a decision-support tool that provides a scam-likelihood score and advisory to the end user, and does not autonomously block transactions, freeze accounts, or take any irreversible action on the user's behalf. The user retains full control over any action taken.

Section 4: Operations Management				
4.	Did your project team adopt best practices in data management and protection in dealing with data owned by the customer / project owner?		☒ YES ☐ NO ☐ NA	Initiation The dataset used was sourced from Kaggle and is fully synthetic, so no real individuals, wallets, or platforms are involved, which minimises privacy risk. Data is uploaded and versioned in S3 using SHA256-based versioning (Notebook 01), and the production design does not permanently store user-submitted messages, inputs are processed only for inference and not saved.
5.	Is the dataset used relevant?		☒ YES ☐ NO ☐ NA	Implementation The dataset used matches the problem, 10,466 crypto-related messages labelled scam or legitimate, which mirrors the real-world channels scammers use to reach victims.

6.	Where personal data is involved, is it collected for the intended purposes?	☐ YES ☐ NO ☒ NA	Implementation	For the dataset used in training, there is no personal data involved. It consists of synthetically generated message text, platform labels, and scam/legitimate labels only, with no real user identities, wallet addresses, or contact details tied to actual individuals. For the deployed application itself, no personal data is collected or saved either. User-submitted messages pasted into the Streamlit app are processed to generate a scam-likelihood prediction and are not stored or retained. This design choice avoids introducing personal data handling risk even though users may paste messages containing wallet addresses, phone numbers, or other personal details.
7.	Is the dataset used well-structured and in a machine understandable form?	☒ YES ☐ NO ☐ NA	Implementation	The dataset is a clean, structured tabular CSV (id, platform, text, label) with 0 missing values and 0 duplicate messages (Notebook 01). It was further processed into a machine-understandable form through text cleaning, 15 engineered indicator features, and TF-IDF vectorisation.
8.	[Relevant only in limited scenarios:] If the dataset used has been joined from multiple datasets, were the extraction, transformation and other relevant operations performed correctly?	☐ YES ☐ NO ☒ NA	Implementation	The dataset was obtained as a single, pre-assembled dataset from Kaggle and was not joined or merged from multiple separate data sources. No cross-dataset extraction or transformation operations were required.
9.	Did your project team identify and propose solutions to mitigate unintended biases in the dataset used for the AI model, especially omission bias and stereotype bias?	☒ YES ☐ NO ☐ NA	Implementation	We identified and checked for several bias risks. Class imbalance was assessed and found to be mild, so it did not require explicit resampling or class-weighting techniques. Platform-level scam rates were also checked and found roughly consistent across all 6 platforms, confirming that platform is not a meaningful source of bias in the dataset. We also flagged that the dataset is synthetically generated, which is itself a residual bias and generalisation concern, and documented this as a limitation requiring validation on real-world data before production use.
10.	Did your organization use different datasets for training, testing and validation of the AI model?	☒ YES ☐ NO ☐ NA	Implementation	We used a stratified 80/20 train/test split (Notebook 01). The TF-IDF vectoriser is fitted on the training split only to prevent information leakage. For model selection, we additionally used cross-validation during hyperparameter tuning (Notebook 02) rather than a separate static validation

				set, and a quality gate of $\text{AUC-ROC} \geq 0.85$ is enforced before any model is promoted in the Model Registry (Notebook 03).
11.	Can your project team explain how the deployed AI model functions and arrives at a particular prediction?		☒ YES ☐ NO ☐ NA	Implementation Logistic regression with TF-IDF was selected as our final model because it performed best during our trials. A useful side benefit of this choice is that logistic regression is also inherently more interpretable than black-box alternatives, so predictions can be explained through model coefficients and engineered feature values. Feature importance plots are logged via MLflow for the tree-based candidate models as well (Notebook 02), giving a point of comparison. The application also surfaces scam probability rather than a bare binary label, and is designed to display key suspicious indicators detected in the message so users understand why a message was flagged.